

GEF Secure

Centro de Escaneos

Propuesta de página centralizada con detalle de vulnerabilidades

Documento de propuesta técnica: consolidar en una sola pantalla los 3 disparadores de escaneo hoy dispersos entre Inventario y Dashboard, mostrando el detalle real de las vulnerabilidades encontradas — no solo un conteo agregado — reutilizando piezas de backend y frontend que ya existen y ya están probadas en producción.

Índice

- 1. Resumen ejecutivo**
- 2. Estado actual: 3 disparadores, 3 lugares distintos**
- 3. Lo que ya existe y se puede reutilizar sin cambios**
- 4. Arquitectura objetivo**
- 5. Implementación frontend (sin cambios de backend)**
 - 5.1 Página nueva: Scans.tsx
 - 5.2 Ruta y navegación
 - 5.3 Qué se saca de Inventario y Dashboard
- 6. Decisiones de diseño**
- 7. Procedimiento de implementación sugerido**
- 8. Validación (checklist)**
- 9. Fuera de alcance / mejoras futuras**
- 10. Conclusión**

1. Resumen ejecutivo

Este documento propone una página nueva — **Centro de Escaneos** — que centraliza en un solo lugar los 3 disparadores de escaneo (por activo, por entorno, global) que hoy están repartidos entre Inventario y Dashboard, y muestra los resultados con **detalle real por vulnerabilidad** — identificador CVE/GHSA, activo afectado, CVSS, prioridad, estado y una descripción del hallazgo — no un simple conteo agregado. A diferencia de [GEF_Secure_Automatizacion_n8n.pdf](#) y [GEF_Secure_Scan_Alcance.pdf](#) — que documentan trabajo ya implementado y verificado contra Docker real — este es un documento de **propuesta**: el código que sigue está listo para implementarse, pero la página en sí todavía no existe en el repo.

Dos hallazgos de la inspección previa a este documento cambiaron el diseño respecto a una primera versión de esta propuesta:

Hallazgo 1 — no existe vínculo entre un scan puntual y las vulnerabilidades que encontró. Se revisó el esquema real (`init/01-create-tables.sql`): `vulnerabilities_audit` no tiene ninguna columna que la relacione con `scan_reports` (ni `scan_report_id` ni nada equivalente). No hay forma de responder "qué encontró exactamente esta ejecución" sin agregar esa relación al modelo de datos — cambio de esquema real, no solo de backend/frontend, y fuera del alcance de este documento (queda anotado en §9). La solución adoptada: la tabla de resultados no muestra "lo que encontró el último scan" sino **el estado actual completo de las vulnerabilidades**, que se refresca solo al completar cualquier disparo — mismo dato, mismo componente, que ya usa Auditoría/Kanban hoy.

Hallazgo 2 — el detalle ya existe en la base, pero ninguna pantalla lo muestra. La columna `decision` de `vulnerabilities_audit` no está vacía: el nodo de código del workflow de n8n (`jsCode`, sección "MATRIZ DE RIESGO CONTEXTUAL") arma un `action_plan` — una frase concreta por vulnerabilidad, por ejemplo "■■■ PRIORIDAD MÁXIMA: Activo crítico en Producción. Remediación requerida en menos de 24hs." o "■ EMERGENCIA (CISA KEV) en Producción: Explotación activa detectada. AISLAR 'react-app' y aplicar parche v18.2.1." — y ese texto se guarda tal cual en `decision` en cada `INSERT`. Verificado además contra `VulnerabilityCard.tsx` (la tarjeta que usa Auditoría/Kanban): no renderiza `vuln.decision` en ningún lado. Es un dato real, ya generado en cada scan, que hoy no es visible en ninguna pantalla del sistema.

Pieza	Estado
<code>GET /api/vulnerabilities</code>	Ya existe y ya está en producción — es el mismo endpoint que usa <code>Kanban.tsx</code> — 0% nuevo
Datos de detalle (CVE, CVSS, prioridad, estado, <code>decision</code>)	Ya existen en <code>VulnerabilityAuditDTO</code> — 0% nuevo , solo faltaba mostrarlos acá
3 disparadores de scan (activo/entorno/global)	Ya implementados y verificados (GEF_Secure_Scan_Alcance.pdf) — reutilizar tal cual
Hook de polling	<code>useScanPolling()</code> ya soporta múltiples instancias independientes sin modificarlo — reutilizar tal cual
Componente de tabla	<code>Table<T></code> ya es compatible con <code>VulnerabilityAudit</code> sin cambios — reutilizar tal cual
Backend nuevo	Ninguno — este documento es 100% frontend
Página <code>Scans.tsx</code>	No existe — nueva (§5.1)

Este documento no fue validado contra Docker real como los dos anteriores, porque la funcionalidad todavía no está implementada. El código de la sección 5 es la propuesta a implementar; la sección 8 da el checklist para validarlo una vez escrito, siguiendo el mismo criterio de verificación que se usó en los documentos anteriores.

2. Estado actual: 3 disparadores, 3 lugares distintos

Inspeccionando directamente `Inventory.tsx` y `Dashboard.tsx` (código real, no supuesto):

Alcance	Dónde vive hoy	Evidencia
Por activo	Botón por fila, en la tabla de Inventario	<code>Inventory.tsx</code> , columna "Acciones" (<code>handleScan</code>)
Por entorno	Selector + botón, arriba de la tabla de Inventario	<code>Inventory.tsx</code> líneas 192-206 (<code>handleScanEnvironment</code>)
Global	Botón en el header del Dashboard	<code>Dashboard.tsx</code> , prop <code>actions</code> de <code>PageHeader</code>

El único lugar del frontend que hoy muestra detalle de vulnerabilidades es Auditoría/Kanban (`Kanban.tsx` + `VulnerabilityCard.tsx`), pero tiene dos límites relevantes para este documento: no tiene ninguna relación con qué scan encontró cada vulnerabilidad (ver Hallazgo 1, §1), y no muestra el campo `decision` aunque esté guardado (ver Hallazgo 2, §1) — la tarjeta solo expone CVE, CVSS, prioridad y asignado.

Consecuencia práctica. Hoy, para saber si un escaneo por entorno o global encontró algo nuevo, la única señal es el toast que dispara `useScanPolling` en el momento ("Escaneo completo") — sin ningún detalle de qué se encontró. Para ver el detalle hay que navegar a Auditoría y, aun ahí, el plan de acción de cada hallazgo no se ve. Centralizar los disparadores en una pantalla que además muestra ese detalle resuelve ambos problemas de una vez.

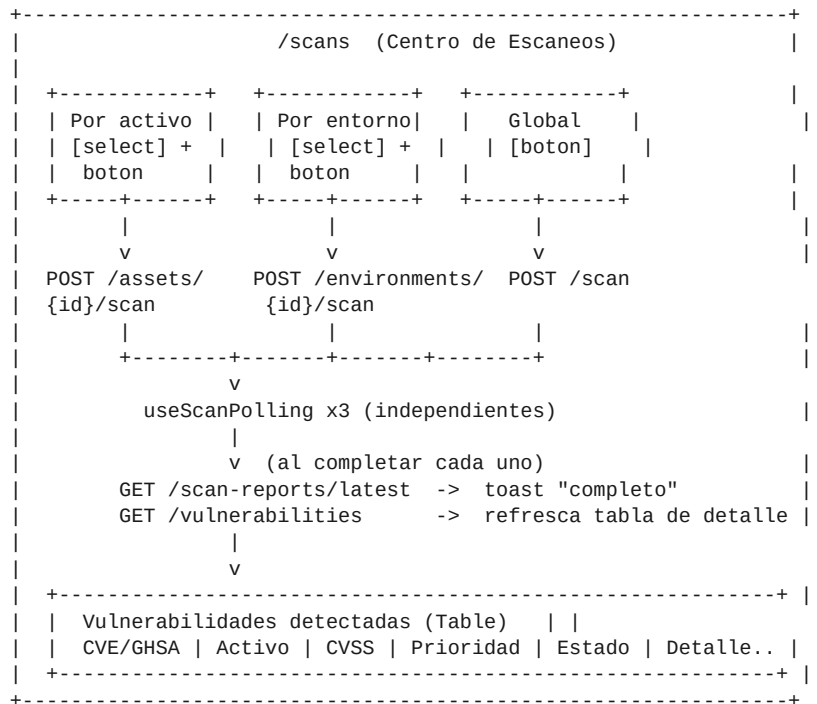
3. Lo que ya existe y se puede reutilizar sin cambios

Antes de proponer código nuevo, esto es lo que ya existe en el proyecto y que el Centro de Escaneos usa tal cual, sin modificar ni un archivo de los que siguen:

Pieza	Por qué se reutiliza sin cambios
<code>assetApi.triggerScan(id)</code> <code>scanApi.triggerEnvironment(id)</code> <code>scanApi.triggerGlobal()</code>	Los 3 disparadores ya existen en <code>services/api.ts</code> y ya funcionan (documentados y verificados en <code>GEF_Secure_Scan_Alcance.pdf</code>) — la página nueva solo los llama desde otro componente.
<code>useScanPolling()</code>	Es un hook de estado propio (usa <code>useState/useRef</code> internos) — cada <code>const x = useScanPolling()</code> en el mismo componente crea una máquina de estados independiente. Se pueden instanciar 3 veces (una por tarjeta de disparo) sin ningún cambio en el hook.
<code>vulnApi.getAll()</code> (GET <code>/api/vulnerabilities</code>)	Es el mismo endpoint que ya consume <code>Kanban.tsx</code> — soporta filtro opcional por <code>?status=</code> (no se usa acá, se trae todo) y devuelve el listado completo con <code>VulnerabilityAuditDTO</code> .
<code>VulnerabilityAudit</code> (tipo)	Ya trae <code>cveId</code> , <code>ghsaId</code> , <code>cvss</code> , <code>priority</code> , <code>status</code> , <code>asset</code> , <code>decision</code> y <code>detectedAt</code> — exactamente los campos que necesita la tabla de resultados, sin agregar ni un campo nuevo.
<code>Table<T></code>	Su firma es <code>Table<T extends { id?: number string }></code> y <code>VulnerabilityAudit</code> sí tiene <code>id</code> — encaja sin ninguna de las salvedades que hacían falta para <code>ScanReport</code> en una versión anterior de este documento.

En términos de superficie de cambio: **no hay backend nuevo**. Todo el frontend nuevo es un archivo de página (§5.1) más una línea de ruta y una de navegación (§5.2). No se toca ningún archivo de `n8n`, ningún workflow, ninguna tabla de base de datos.

4. Arquitectura objetivo



Los 3 disparadores conviven en la misma pantalla pero son **independientes entre sí** — no hay ninguna coordinación necesaria porque cada uno usa su propia instancia de `useScanPolling` y su propio estado de "disparando". Disparar un scan de entorno mientras un scan de activo sigue en curso no cancela ni interfiere con el otro (a diferencia de usar una sola instancia compartida del hook, que sí se pisarían — ver §6). Los tres, al completar, refrescan la *misma* tabla de abajo — no hay una tabla por tipo de scan.

5. Implementación frontend (sin cambios de backend)

5.1 Página nueva: Scans.tsx

Archivo nuevo `frontend/src/pages/Scans.tsx`, siguiendo el mismo patrón de composición que ya usan `Inventory.tsx` y `Settings.tsx` (PageHeader + card + Table, toasts para feedback). No hace falta agregar ningún tipo ni ningún método al API client — `vulnApi` y `VulnerabilityAudit` ya existen y ya se importan igual en `Kanban.tsx`:

```
import { useEffect, useState, useCallback } from 'react'
import { RefreshCw } from 'lucide-react'
import toast from 'react-hot-toast'
import { assetApi, environmentApi, vulnApi, scanApi } from '../services/api'
import type { Asset, Environment, VulnerabilityAudit } from '../types'
import PageHeader from '../components/PageHeader'
import Table, { Column } from '../components/Table'
import { useScanPolling } from '../hooks/useScanPolling'

const PRIORITY_BADGE: Record<string, string> = {
  CRITICAL: 'bg-[#bd1e1e]/20 text-[#bd1e1e] border border-[#bd1e1e]/40',
  HIGH:     'bg-[#f95c5c]/15 text-[#f95c5c] border border-[#f95c5c]/30',
  MEDIUM:  'bg-[#f9f15c]/15 text-yellow-200 border border-[#f9f15c]/30',
  LOW:      'bg-[#44a024]/15 text-[#44a024] border border-[#44a024]/30',
}

const STATUS_BADGE: Record<string, string> = {
  DETECTADA: 'bg-slate-600/30 text-slate-300 border border-slate-500/30',
  EN_ANALISIS: 'bg-amber-600/20 text-amber-400 border border-amber-600/30',
  RESUELTA: 'bg-emerald-600/20 text-emerald-400 border border-emerald-600/30',
}

export default function Scans() {
  const [assets, setAssets] = useState<Asset[]>([])
  const [environments, setEnvironments] = useState<Environment[]>([])
  const [vulns, setVulns] = useState<VulnerabilityAudit[]>([])
  const [loadingVulns, setLoadingVulns] = useState(true)

  const [assetToScan, setAssetToScan] = useState<number | ''>('')
  const [envToScan, setEnvToScan] = useState<number | ''>('')
  const [scanningAsset, setScanningAsset] = useState(false)
  const [scanningEnv, setScanningEnv] = useState(false)
  const [scanningGlobal, setScanningGlobal] = useState(false)

  // 3 instancias independientes -- ver seccion 3, no hace falta tocar el hook
  const activoPoll = useScanPolling()
  const entornoPoll = useScanPolling()
  const globalPoll = useScanPolling()

  // repository.findAll() no garantiza orden -- se ordena en el cliente
  const loadVulns = useCallback(() => {
    setLoadingVulns(true)
    vulnApi.getAll()
      .then(r => {
        const sorted = [...r.data].sort((a, b) =>
          new Date(b.detectedAt).getTime() - new Date(a.detectedAt).getTime())
        setVulns(sorted)
      })
      .catch(() => toast.error('Error al cargar las vulnerabilidades'))
      .finally(() => setLoadingVulns(false))
  }, [])

  useEffect(() => {
    loadVulns()
    assetApi.getAll()
  })
}
```



```

        .then(r => setAssets(r.data))
        .catch(() => toast.error('Error al cargar activos'))
environmentApi.getAll()
    .then(r => setEnvironments(r.data))
    .catch(() => toast.error('Error al cargar entornos'))
}, [loadVulns])

const handleScanAsset = async () => {
    if (!assetToScan) return
    const asset = assets.find(a => a.id === assetToScan)
    if (!asset) return
    setScanningAsset(true)
    try {
        await assetApi.triggerScan(asset.id)
        toast(`Escaneando "${asset.name}"...`)
        activoPoll.start('ACTIVO', asset.name, () => {
            toast.success(`Escaneo de "${asset.name}" completo`)
            loadVulns()
        })
    } catch (err: unknown) {
        toast.error(
            err instanceof Error ? err.message : 'Error al disparar el escaneo')
    } finally {
        setScanningAsset(false)
    }
}

const handleScanEnvironment = async () => {
    if (!envToScan) return
    const env = environments.find(e => e.id === envToScan)
    if (!env) return
    setScanningEnv(true)
    try {
        await scanApi.triggerEnvironment(env.id)
        toast(`Escaneando entorno "${env.name}"...`)
        entornoPoll.start('ENTORNO', env.name, () => {
            toast.success(`Escaneo de "${env.name}" completo`)
            loadVulns()
        })
    } catch (err: unknown) {
        toast.error(err instanceof Error
            ? err.message : 'Error al disparar el escaneo del entorno')
    } finally {
        setScanningEnv(false)
    }
}

const handleScanGlobal = async () => {
    setScanningGlobal(true)
    try {
        await scanApi.triggerGlobal()
        toast('Escaneo global disparado...')
        globalPoll.start('GLOBAL', 'TODOS', () => {
            toast.success('Escaneo global completo')
            loadVulns()
        })
    } catch (err: unknown) {
        toast.error(err instanceof Error
            ? err.message : 'Error al disparar el escaneo global')
    } finally {
        setScanningGlobal(false)
    }
}

const columns: Column<VulnerabilityAudit>[] = [
    {
        key: 'cveId', label: 'CVE / GHSA',
        render: (_v, row) => (

```

```

      <span className="font-mono text-xs">
        {row.cveId || row.ghsaId || 'Sin identificador'}
      </span>
    ),
  },
  { key: 'asset', label: 'Activo' },
  {
    key: 'cvss', label: 'CVSS',
    render: v => (
      <span className="font-mono text-xs">{v ? String(v) : '-'}</span>
    ),
  },
  {
    key: 'priority', label: 'Prioridad',
    render: v => (
      <span className={`badge ${PRIORITY_BADGE[String(v)] ?? ''}`}>
        {String(v) || 'SIN PRIORIDAD'}
      </span>
    ),
  },
  {
    key: 'status', label: 'Estado',
    render: v => (
      <span className={`badge ${STATUS_BADGE[String(v)] ?? ''}`}>
        {String(v)}
      </span>
    ),
  },
  {
    key: 'decision', label: 'Detalle',
    render: v => (
      <span className="text-xs text-slate-400 line-clamp-2 max-w-xs">
        {v ? String(v) : 'Sin detalle'}
      </span>
    ),
  },
  {
    key: 'detectedAt', label: 'Detectado',
    render: v => new Date(String(v)).toLocaleString('es-AR'),
  },
]

return (
  <div className="animate-fade-in">
    <PageHeader
      title="Centro de Escaneos"
      subtitle="Disparar y revisar escaneos por activo, entorno o global"
    />

    <div className="grid grid-cols-3 gap-4 mb-6">
      <div className="card">
        <p className="text-xs font-semibold text-slate-400 mb-3 uppercase tracking-wider">
          Por activo
        </p>
        <select
          className="input mb-2"
          value={assetToScan}
          onChange={e =>
            setAssetToScan(e.target.value ? Number(e.target.value) : '')
          }
        >
          <option value="">Seleccionar activo...</option>
          {assets.map(a => (
            <option key={a.id} value={a.id}>{a.name}</option>
          ))}
        </select>
        <button
          onClick={handleScanAsset}
          disabled={!assetToScan || scanningAsset}

```

```

        className="btn-primary w-full"
      >
        {scanningAsset || activoPoll.state === 'running'
          ? 'Escaneando...' : 'Escanear activo'}
      </button>
    </div>

    <div className="card">
      <p className="text-xs font-semibold text-slate-400 mb-3 uppercase tracking-wider">
        Por entorno
      </p>
      <select
        className="input mb-2"
        value={envToScan}
        onChange={e =>
          setEnvToScan(e.target.value ? Number(e.target.value) : '')}
      >
        <option value="">Seleccionar entorno...</option>
        {environments.map(env => (
          <option key={env.id} value={env.id}>{env.name}</option>
        ))}
      </select>
      <button
        onClick={handleScanEnvironment}
        disabled={!envToScan || scanningEnv}
        className="btn-primary w-full"
      >
        {scanningEnv || entornoPoll.state === 'running'
          ? 'Escaneando...' : 'Escanear entorno'}
      </button>
    </div>

    <div className="card flex flex-col justify-between">
      <div>
        <p className="text-xs font-semibold text-slate-400 mb-3 uppercase tracking-wider">
          Global
        </p>
        <p className="text-xs text-slate-500 mb-3">
          Todos los entornos, todos los activos.
        </p>
      </div>
      <button
        onClick={handleScanGlobal}
        disabled={scanningGlobal}
        className="btn-primary w-full"
      >
        {scanningGlobal || globalPoll.state === 'running'
          ? 'Escaneando...' : 'Escanear todo'}
      </button>
    </div>
  </div>

  <div className="flex justify-between items-center mb-4">
    <p className="text-sm text-slate-500">Vulnerabilidades detectadas</p>
    <button
      onClick={loadVulns}
      disabled={loadingVulns}
      className="btn-ghost"
    >
      <RefreshCw size={13} className={loadingVulns ? 'animate-spin' : ''} />
      Recargar
    </button>
  </div>

  <div className="card p-0 overflow-hidden rounded-xl border border-surface-600">
    <Table
      columns={columns}
      data={vulns}
      loading={loadingVulns}
    >

```

```

        emptyMessage="No se encontraron vulnerabilidades"
      />
    </div>
  </div>
)
}

```

5.2 Ruta y navegación

App.tsx — agregar la ruta:

```

import Scans from './pages/Scans'
// ...
<Route path="scans" element={ <Scans /> } />

```

Sidebar.tsx — agregar el link inmediatamente debajo de Dashboard (el pedido puntual de este documento: la sección de escaneos va "abajo de Dashboard" en el menú):

```

import {
  LayoutDashboard, Package, KanbanSquare,
  Settings, ShieldCheck, Activity, ScanLine,
} from 'lucide-react'

const links = [
  { to: '/dashboard', icon: LayoutDashboard, label: 'Dashboard' },
  { to: '/scans', icon: ScanLine, label: 'Escaneos' }, // nuevo
  { to: '/inventory', icon: Package, label: 'Inventario' },
  { to: '/kanban', icon: KanbanSquare, label: 'Auditoría' },
  { to: '/settings', icon: Settings, label: 'Config' },
]

```

ScanLine ya se usa como ícono en **Inventory.tsx** (botón de scan por fila) — se reutiliza el mismo ícono para que el usuario asocie visualmente ambos lugares con la misma acción.

5.3 Qué se saca de Inventario y Dashboard

Para que "centralizar" sea real y no una copia más de los mismos controles, se sacan de sus ubicaciones actuales los dos disparadores que no tienen un lugar natural propio (entorno y global) y se dejan **solo** en el Centro de Escaneos:

Alcance	Se saca de	Se queda en	Motivo
Por entorno	Inventory.tsx (bloque líneas 192-206, y los estados envToScan/scanningEnv asociados)	Solo Centro de Escaneos	No tenía una fila propia en Inventario — estaba flotando arriba de la tabla, sin relación visual clara con ninguna fila
Global	Dashboard.tsx (prop actions del PageHeader , y el estado scanningAll asociado)	Solo Centro de Escaneos	Mismo motivo — el Dashboard queda como pantalla de solo lectura de KPIs, sin acciones mezcladas

Por activo	No se saca de ningún lado	Botón por fila en Inventario y selector en Centro de Escaneos	El botón por fila es conveniencia contextual real (escanear el activo que se está mirando en ese momento) — duplicar este caso concreto no reintroduce el problema que motiva este documento, porque ambos disparan el mismo endpoint y no hay dos fuentes de verdad distintas
------------	---------------------------	---	--

Por qué no se saca también el botón por fila de Inventario. El objetivo de este documento es juntar los disparadores que hoy están *sueltos, sin contexto* (entorno y global). El botón por fila de un activo específico sí tiene contexto — está al lado de los demás datos de ese activo (último scan, criticidad, entorno). Sacarlo empeoraría la experiencia sin resolver ningún problema real.

6. Decisiones de diseño

Decisión	Justificación
Mostrar el estado actual de <code>vulnerabilities_audit</code> , no "lo que encontró este scan"	Explicado en el Hallazgo 1 (§1): no existe la relación en el modelo de datos hoy. Mostrar el estado completo, refrescado al terminar cualquier disparo, da la misma sensación práctica ("veo lo que se encontró") sin inventar una relación que no está.
3 instancias de <code>useScanPolling</code> , no 1 compartida	El hook guarda un solo <code>timerRef</code> y un solo <code>state</code> por instancia. Si las 3 tarjetas compartieran una instancia, disparar un scan de entorno mientras el de activo sigue en curso llamaría a <code>stop()</code> internamente (línea 30 del hook) y perdería el polling del primero sin avisar. Con 3 instancias, cada tarjeta es independiente por diseño, sin tocar el hook.
Ordenar por <code>detectedAt</code> en el cliente, no en el backend	<code>VulnerabilityAuditService.findAll()</code> llama a <code>repository.findAll()</code> sin ningún <code>ORDER BY</code> — el orden no está garantizado. Como este documento evita tocar backend (§3), se ordena el array ya en el cliente en vez de agregar un método de repositorio nuevo.
<code>decision</code> truncado con <code>line-clamp-2</code> , no completo	El texto del <code>action_plan</code> puede ser largo (ver ejemplos del Hallazgo 2, §1). <code>line-clamp-1</code> ya es un patrón usado en el proyecto (<code>Settings.tsx</code> , columna de mensaje de error) — acá se usa <code>line-clamp-2</code> porque las frases de <code>action_plan</code> son más largas que un mensaje de error típico.
Sin paginación en la tabla de vulnerabilidades	Consistente con como ya funciona <code>Kanban.tsx</code> hoy (trae todo de una, sin paginar) — no se introduce una inconsistencia de UX entre las dos pantallas que muestran el mismo tipo de dato.

7. Procedimiento de implementación sugerido

1. **Página base:** §5.1 (`Scans.tsx` completo) + §5.2 (ruta y link de sidebar). En este punto la página ya es usable en paralelo con los controles viejos de Inventario y Dashboard — sirve para probar todo antes de sacar nada.
2. **Limpieza:** §5.3, recién después de confirmar que el Centro de Escaneos funciona: sacar el bloque de scan por entorno de `Inventory.tsx` y el botón global de `Dashboard.tsx`.

No hay un paso de backend porque no hace falta ninguno (§3) — todo el procedimiento es frontend.

8. Validación (checklist)

Verificación		Cómo comprobarlo
■	La tabla muestra detalle real, no un conteo	Cada fila debe traer CVE/GHSA, activo, CVSS, prioridad, estado y el texto de <code>decision</code> — no solo números agregados
■	Las 3 tarjetas disparan scans independientes sin pisarse	Disparar activo y entorno casi al mismo tiempo — ambos toasts de "completo" deben aparecer, ninguno debe quedar en "Escaneando..." para siempre
■	La tabla se refresca sola al completar un scan	Disparar cualquier scan sobre un activo con hallazgos nuevos y confirmar que aparecen sin recargar la página
■	El orden es el correcto (más reciente primero)	Comparar la primera fila contra <code>SELECT MAX(detected_at) FROM vulnerabilities_audit</code>
■	Estado vacío se ve bien en un ambiente recién levantado	Contra un volumen sin vulnerabilidades, la tabla debe mostrar "No se encontraron vulnerabilidades", no un error ni una tabla en blanco
■	<code>decision</code> nulo no rompe la fila	Si alguna fila tiene <code>decision</code> vacío (creada manualmente, no por n8n), debe mostrar "Sin detalle", no <code>undefined</code> ni una celda rota
■	Inventario y Dashboard siguen funcionando tras la limpieza de §5.3	El botón por fila de Inventario sigue andando; Dashboard carga sin el botón "Escanear todo" y sin errores de import rotos (<code>useScanPolling</code> sin usar, <code>scanApi</code> sin usar)

9. Fuera de alcance / mejoras futuras

- **Vincular cada vulnerabilidad al scan que la encontró** — requiere agregar `scan_report_id` a `vulnerabilities_audit` (migración de esquema) y modificar el workflow de n8n para setearlo al insertar (capturar el id generado por el `INSERT` de `scan_reports` y pasarlo al `INSERT` de vulnerabilidades de la misma ejecución). Es la forma correcta de responder "qué encontró exactamente este scan", pero es un cambio de base de datos + workflow, no de back/frontend — se documenta acá para que quede como próximo paso posible, no se implementa ahora.
- **Filtros en la tabla** (por estado, por prioridad, por activo) — hoy se trae todo, igual que `Kanban.tsx`; si el volumen crece, agregar filtros es un cambio aislado a `Scans.tsx` reutilizando el `?status=` que el endpoint ya soporta.
- **Ver el texto completo de `decision`** — hoy se trunca a 2 líneas; un modal o expandir la fila al hacer click es una mejora simple si en la demo se nota que se corta información importante.
- **Actualización en vivo de la tabla mientras hay un scan en curso** — no hay señal intermedia de progreso desde n8n hoy, el mismo argumento que ya dieron los dos documentos anteriores sobre por qué no se justifica WebSocket/SSE para este volumen de eventos.
- **Cancelar un scan en curso** — el workflow de n8n no expone ningún mecanismo de cancelación; el polling puede dejar de esperar (timeout), pero el scan real sigue corriendo del lado de n8n.

10. Conclusión

Este documento propone consolidar en una sola pantalla lo que hoy son 3 puntos de disparo dispersos, y mostrar junto a ellos el detalle real de lo encontrado — no un conteo, sino CVE, activo, CVSS, prioridad, estado y el plan de acción que n8n ya genera por cada hallazgo pero que hoy no se ve en ningún lado. El costo de implementación es bajo porque no hace falta tocar el backend: el endpoint, el tipo y los datos ya existen y ya están probados en producción vía Auditoría/Kanban; lo único genuinamente nuevo es un archivo de página que los combina con los 3 disparadores ya verificados en el documento anterior.

Estado al aplicar este documento: un lugar único para disparar cualquiera de los 3 tipos de escaneo y ver el detalle real de las vulnerabilidades encontradas, sin duplicar lógica de negocio ni tocar n8n, Docker, backend, ni el esquema de base de datos.